0010. Regular Expression Matching

Hard · 2-D Dynamic Programming

leetcode.com/problems/regular-expression-matching

Problem

- Implement full-string matching with '.' (any single char) and '' (zero or more of the preceding element).

Intuition

- Dp[i][j] = does s[0..i) match p[0..j)?
- If p[j-1] is '': either use zero copies (dp[i][j-2]) or, if s[i-1] matches p[j-2], consume one char of s (dp[i-1][j]).
- Otherwise: chars match (or '.') and dp[i-1][j-1].

Brute Force

- Plain recursion on the two branches of '': exponential.

Optimal Approach

- Dp[0][0] = true; dp[0][j] = p[j-1] == '' && dp[0][j-2] (patterns like ab match empty).
- Fill the table with the rules above.

Worked Example

```
s = "aab", p = "cab".

"c*" matches empty (zero c's)
"a*" consumes "aa"
"b" matches "b"
dp[3][5] = true
```

Java Solution

```java
public class RegularExpressionMatching {
    public boolean isMatch(String s, String p) {
        int m = s.length(), n = p.length();
        boolean[][] dp = new boolean[m + 1][n + 1];
        dp[0][0] = true;
        for (int j = 2; j <= n; j++) dp[0][j] = p.charAt(j - 1) == '*' && dp[0][j - 2];
        for (int i = 1; i <= m; i++)
            for (int j = 1; j <= n; j++) {
                char pc = p.charAt(j - 1);
                if (pc == '*') {
                    dp[i][j] = dp[i][j - 2] || (matches(s.charAt(i - 1), p.charAt(j - 2)) && dp[i - 1][j]);
                } else {
                    dp[i][j] = matches(s.charAt(i - 1), pc) && dp[i - 1][j - 1];
                }
            }
        return dp[m][n];
    }

    private boolean matches(char sc, char pc) { return pc == '.' || pc == sc; }
}
```

Complexity

- Time: O(m n). Space: O(m n).

Pitfalls

- '' applies to the previous element, so look at p[j-2].
- Matching must cover the whole string, not a substring.